VeriVLA: Adaptive Test-Time Verification for Flow-Based VLAs with Latent World Models

Xueli Sun [1] **Yuxiang Lu** [1] **Yantong Liu** [1] **Xianzhe Fan** [1] **Xi Chen** [1] **Zhe Liu** [1 †] **Hengshuang Zhao** [1 *]

Abstract

Flow-based Vision-Language-Action (VLA) policies are inherently stochastic: different noise seeds can produce action chunks of varying quality from the same observation. At test time, however, the policy typically commits to a single sample without an explicit estimate of its quality. In fine-grained manipulation, executing a suboptimal action can irreversibly disrupt task progress and lead to failure. To exploit policy stochasticity, we introduce a training-free two-axis trigger that uses two initial samples to measure within-chunk temporal variation and cross-sample variance, detecting the critical states. Once triggered, a JEPA-based latent world model ranks candidate action chunks by predicting action-conditioned future latent states and the corresponding changes in task progress. By operating in latent space, the verifier avoids pixel reconstruction and enables efficient and more accurate ranking. Across three real-world tasks (*i.e.*, pen insertion, pipetting, and apple placement), our method raises the average completion score from 50.8% to 65.5%, while adding 326 ms for sampling and verification per triggered decision on a single RTX 4090. Consistent gains are also observed in simulation, where the average success rate rises from 66.4% to 73.6% in the clean setting of RoboTwin 2.0, from 62.8% to 69.6% in its randomized setting, and from 96.9% to 97.6% on LIBERO. Code and demos: https://github.com/shirley430316/VeriVLA.

1. Introduction

Vision-Language-Action (VLA) models have emerged as a promising paradigm towards general robotic control (Brohan et al., 2023; Kim et al., 2024; Black et al., 2025a; Intelligence et al., 2025). In the pretraining stage, the field has evolved from VLMs autoregressively generating over discretized action tokens (Zitkovich et al., 2023; Kim et al., 2024; Pertsch et al., 2025) to more recent flow-matching action experts coupled with pretrained VLMs (NVIDIA et al., 2025; Black et al., 2025a; Intelligence et al., 2025), enabling scalable learning from diverse action trajectories. The paradigm shift was foreshadowed by diffusion-based visuomotor policies (Chi et al., 2023), which established iterative denoising as an effective means of learning multi-modal (*i.e.*, multi-peaked) action distributions. However, the stochasticity that benefits learning also introduces a unique challenge at deployment.

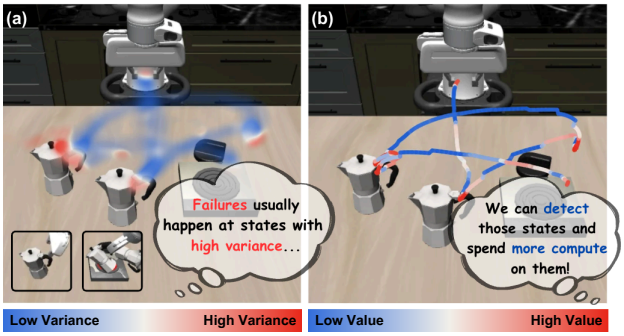


Figure 1. **(a) Policy variance in the LIBERO environment.** Repeated sampling reveals higher action variance in contact-rich regions, where task failures are most likely to occur. **(b) Values of the proposed trigger.** The trigger value aligns with task difficulty and the policy's internal uncertainty. These observations motivate triggering repeated sampling and verification at critical decision steps.

Given the same observation and instruction, different noise seeds yield samples of different quality during inference. However, the flow-based VLA model cannot distinguish which action chunk is most likely to succeed. During execution, a robot must commit to one sampled action chunk, which may irreversibly change the physical world. A good action chunk advances the task, while a poor one could directly lead to failure or accumulate errors that drift the trajectory out of distribution (Ross et al., 2011). We empirically show that the policy variance is non-uniform along the task, and failures tend to happen in high-variance segments of the action trajectory, as in Figure 1(a). What if we could detect these critical moments before committing to an irreversible action, and briefly "rewind" the decision by spending additional computation? This calls for two com-

*Corresponding author. †Project lead. [1]The University of Hong Kong.

Preprint.

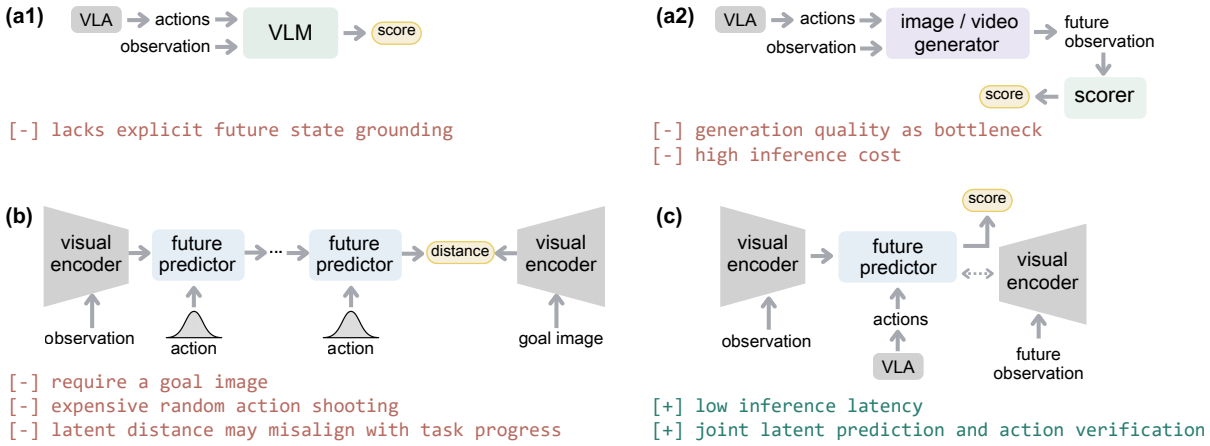


Figure 2. **Conceptual comparison between different paradigms. (a1) Sampling-and-selection with VLM:** Directly predict the delta progress with the current observation and the action chunk produced by VLAs. **(a2) Sampling-and-selection with generative models:** Extract the delta progress after generating the estimated future. **(b) Model Predictive Control (MPC) with latent world models:** Execute action(s) that minimize the latent distance with the goal image. **(c) Latent world model verifier (ours):** Estimate the future latent and the delta progress in a unified latent space.

ponents: a trigger that decides *when* the states are critical and additional computation is worthwhile, and a verifier that determines *which* sampled action should be executed.

To answer the first question, we start with the policy's uncertainty. As shown in Figure 1(a), when the end-effector is close to a contact-rich interaction region, the trajectory follows a more complex motion (Pang et al., 2023), and internally, the policy uncertainty increases. We therefore combine temporal action variation and cross-sample uncertainty into a training-free two-axis trigger. Computed with two sampled action chunks, the trigger closely tracks the contact information from the environment and aligns well with the policy's intrinsic uncertainty, as shown in Figure 1(b). We therefore use the trigger to activate repeated sampling and verification at critical decision steps.

Once repeated sampling is activated, the second question arises: how should candidate actions be evaluated? We argue that latent world models are a natural fit. The verifier evaluates the quality of action chunk samples by predicting the progress or relative ranking (Kwok et al., 2025; Fan et al., 2026). Therefore, the model must understand world dynamics and represent the future states resulting from executing the action chunk. In addition, verification is a low-dimensional problem: how much an action chunk advances the task is a scalar judgment, and most camera pixels are irrelevant to it. As a result, pixel-level world models have two pitfalls: (1) the image or video generation incurs substantial latency, making it difficult to deploy at real-time control frequencies; and (2) they solve the harder pixel generation before the easier progress estimation, making the quality of downstream verification unnecessarily dependent on generation fidelity. We compare these paradigms in Figure 2. This motivates verification in a latent space, which compresses the pixel observation into compact and dynamic-rich features, where we can jointly predict the latent states and task progress.

Building on these observations, we recast action randomness not as a hazard to be suppressed but as a resource to select from, with the latent world model as a verifier. Specifically, we claim the following contributions:

1. **A latent world model as a verifier.** We exploit the action-conditioned latent representation produced by a pretrained latent world model to estimate the progress of candidate action chunks.

2. **An empirical characterization of policy stochasticity in flow-based VLAs and the two-axis trigger.** We characterize policy stochasticity. Leveraging the temporal action variation and the cross-sampling variance, we propose a two-axis trigger that is action-based and training-free. It detects the critical states and reduces inference costs while preserving the benefits of test-time selection.

3. **A simple training, evaluation, and real-robot deployment pipeline.** On real-world manipulation tasks, each triggered round of repeated sampling and verification adds 326 ms of latency, excluding the 14 ms trigger overhead. Our method improves the average completion score by 14.7 percentage points over the base policy, outperforming baselines, including VLM-based verifiers and pixel-level world model verifiers.

2. Related Work

Test-time Enhancement for VLAs. Existing approaches can be broadly categorized according to whether the VLA policy is updated at deployment time. Test-time training methods, such as EVOLVE-VLA (Bai et al., 2025) and TT-VLA (Liu et al., 2026), perform online policy optimization during deployment using dense reward signals. Rather than relying on external reward estimation, T^2VLA (Chen et al., 2026b) derives intrinsic rewards from the policy's own confidence to enable self-supervised adaptation.

Another line of work keeps the pretrained VLA policy frozen and improves action quality through either gradient-free selection or gradient-based augmentation. Gradient-free methods (Xu et al., 2026) generally adopt a repeated-sampling-and-selection strategy where a model-based verifier evaluates candidate actions. RoboMonkey (Kwok et al., 2025) uses a VLM-based verifier to select the optimal action, demonstrating a test-time scaling law of autoregressive VLAs. GPC-RANK (Qi et al., 2026) employs a lightweight pixel-generation model with a downstream estimator to rank candidate actions. DreamAvoid (Fan et al., 2026) combines a pretrained foundation-level, action-conditioned video generation model with an estimator and introduces a vision-based trigger to reduce computational overhead. In contrast, gradient-based methods such as Guided Action Flow (Yang et al., 2026) backpropagate gradients from the verifier score to refine sampled actions.

In this work, we follow the repeated-sampling-and-selection paradigm. Unlike prior verifier-based methods, which rely on VLMs or pixel-level world models followed by downstream signal extraction, our approach uses a latent world model to jointly predict dynamics and estimate task-specific progress, synergizing with an action-based trigger to allocate computation adaptively. We compare more test-time verification methods in Appendix A.

Latent World Models for Robotics. Building on advances in visual representation learning, prior research on latent world models has developed along two main directions. The first line of work (Sobal et al., 2025; Maes et al., 2026; Zhang et al., 2026c), represented by DINO-WM (Zhou et al., 2025) and V-JEPA 2-AC (Assran et al., 2025), utilizes pretrained visual encoders and trains dynamics predictors to perform zero-shot planning in latent space. Given a goal image, these methods employ search-based algorithms to identify suitable action proposals. The second direction combines latent world models with VLAs. VLA-JEPA (Sun et al., 2026) and Being-H0.7 (Luo et al., 2026) incorporate the prediction objective of latent world models into VLA training, while LaWAM (Chen et al., 2026a) introduces a latent subgoal as an additional conditioning signal. These integrated approaches improve overall manipulation performance and generalization.

3. Method

3.1. Problem Formulation

In the standard VLA setting, at each decision step t, a policy π_θ generates an action chunk with current observation o_t, language instruction l, and robot proprioceptive state s_t:

$$a_{t:t+H} \sim \pi_\theta(\cdot \mid o_t, l, s_t), \tag{1}$$

where $a_{t:t+H}$ denotes H consecutive actions, from timestep t to $t+H-1$. Recent VLAs commonly employ flow-matching-based action generation (Intelligence et al., 2025), which takes random Gaussian noise as input. This noise dependence is typically omitted from the policy notation. We make this dependence explicit:

$$a^{(i)}_{t:t+H} \sim \pi_\theta(\cdot \mid o_t, l, s_t, N_i), \tag{2}$$

where N_i denotes the random seed used to generate the Gaussian noise, with a distinct seed for each sampling index i. Repeated sampling therefore produces a set of diverse action chunks $\{a^{(i)}_{t:t+H}\}_{i=1}^{K}$.

Executing an action chunk changes task progress; therefore we define the quality of an action chunk as its change in progress, referred to as *delta progress*,

$$\Delta p = p_{t+H} - p_t, \tag{3}$$

where p_t is the task progress at timestep t. Compared with absolute progress, delta progress matches the effect of the action chunk, making it a more direct and intuitive target for action evaluation (Chen et al., 2025).

3.2. Overview

As illustrated in Figure 3, our framework comprises two tightly related components: Two-Axis Trigger and Latent World Model Verifier. Based on initial action chunk samples, the trigger identifies decision steps at which the policy is likely to produce unreliable actions and activates verification only in these challenging situations.

When the trigger is inactive, no additional action chunks are sampled, and the initial sample $a^{(1)}_{t:t+H}$ is executed directly. When the trigger is activated, the policy generates additional samples to expand the candidate action pool. The verifier V_ϕ then predicts the delta progress associated with each candidate:

$$\left(\hat{z}^{(i)}_{t+H}, \Delta\hat{p}^{(i)}\right) = V_\phi\left(s_t, a^{(i)}_{t:t+H}, o_t; q_l\right), \tag{4}$$

where q_l is the learned query token for the instructed task, $\hat{z}^{(i)}_{t+H}$ denotes the predicted visual latent at time step $t+H$, and $\Delta\hat{p}^{(i)}$ denotes the estimated delta progress of the sample i. The action chunk with the largest estimated delta progress is subsequently executed.

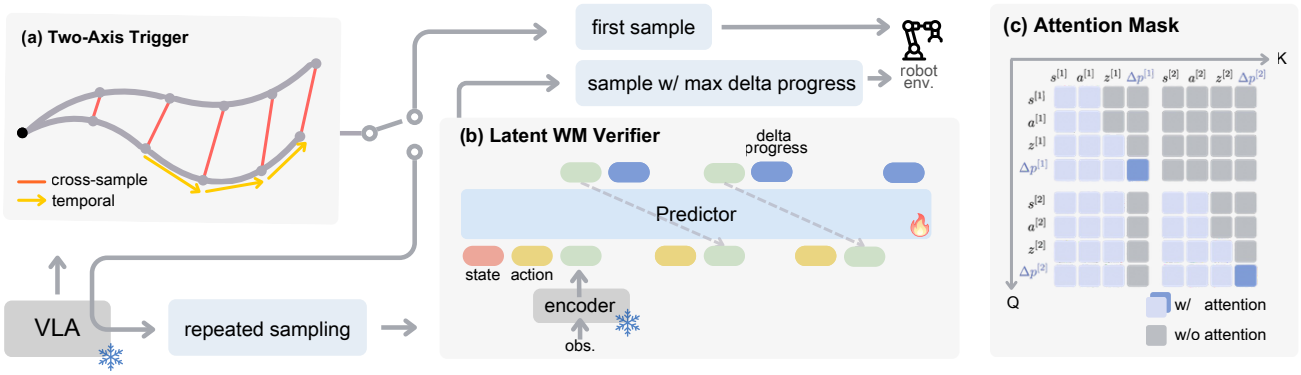


Figure 3. **Overall pipeline. (a) Two-Axis Trigger.** The VLA policy generates two initial action chunk samples, based on which the two-axis trigger decides whether to activate repeated sampling. **(b) Latent World Model Verifier.** If triggered, the verifier evaluates K candidate chunks and selects the one with the highest estimated delta progress for execution. The JEPA-based latent world model autoregressively takes states, actions, and observations as input and predicts delta progress in the latent space. **(c) Attention Mask.** The delta progress key is designed to be invisible from other queries to remove possible task-specific shortcuts for world modeling.

3.3. Two-axis Trigger

Following prior work (Hu et al., 2024; Fan et al., 2026) and our empirical observations (shown in Figure 1), an effective trigger under this setting should satisfy two criteria: (C1) activate in contact-rich regions, where manipulation is inherently more complex (Pang et al., 2023) and failures concentrate; and (C2) activate at states where repeated sampling produces diverse candidate actions, providing the verifier with meaningful alternatives to choose from.

At timestep t, we draw k action chunks $\{a^{(i)}_{t:t+H}\}_{i=1}^{k}$ from the VLA policy, where k is chosen to be much smaller than the full sampling budget K used during verification, so that the trigger incurs acceptable overhead. To avoid bias toward fast-moving stages, each action chunk is first normalized by its average action magnitude $m^{(i)}$. We denote the normalized actions by $\tilde{a}^{(i)}_{t+j} = a^{(i)}_{t+j}/m^{(i)}$. The sampled action chunks can be viewed along two complementary axes. The temporal axis of each action chunk captures C1, while the sample axis across repeated action chunks captures C2. Along the temporal axis, we measure the mean squared changes between consecutive actions within each sampled chunk, defined as

$$T_{\text{temp}}(s_t) = \frac{1}{k}\sum_{i=1}^{k}\frac{1}{H-1}\sum_{j=1}^{H-1}\|\tilde{a}^{(i)}_{t+j} - \tilde{a}^{(i)}_{t+j-1}\|^2. \quad (5)$$

Along the sample axis, we adopt the cross-sample variance, as proposed in AdaFlow (Hu et al., 2024), measuring how much the policy diverges across sampled chunks:

$$T_{\text{cross}}(s_t) = \frac{1}{k}\sum_{i=1}^{k}\frac{1}{H}\sum_{j=0}^{H-1}\left\|\tilde{a}^{(i)}_{t+j} - \bar{\tilde{a}}_{t+j}\right\|^2, \quad (6)$$

where $\bar{\tilde{a}}_{t+j} = \frac{1}{k}\sum_{i=1}^{k}\tilde{a}^{(i)}_{t+j}$ is the mean action at step $t+j$ over k samples. In practice, we use $k = 2$ as two samples are sufficient to reflect the cross-sample divergence.

Before deployment in a new environment, we perform a lightweight calibration by rolling out the base policy and computing T_{temp} and T_{cross} at every timestep for a small number of episodes to collect a reference set. The collected statistics form two empirical reference distributions for the environment. During execution, the trigger values $T_{\text{temp}}(s_t)$ and $T_{\text{cross}}(s_t)$ are converted into their corresponding percentiles, $\tau_{\text{temp}}(s_t)$ and $\tau_{\text{cross}}(s_t)$, within their respective reference distributions. The overall trigger score is then defined as

$$\tau(s_t) = \max\big(\tau_{\text{temp}}(s_t),\ \tau_{\text{cross}}(s_t)\big). \quad (7)$$

The trigger fires when $\tau(s_t) > \tau_0$, where τ_0 is the only trigger hyperparameter that decides how strict the trigger is. Since the trigger correlates well with the critical states, calibrating τ_0 is straightforward and a few rollout episodes are sufficient.

3.4. Latent World Model Verifier

When the Two-Axis Trigger is activated, we perform additional sampling to obtain the remaining $K-k$ action chunk candidates, resulting in a total of K candidates $\{a^{(i)}_{t:t+H}\}_{i=1}^{K}$. Unlike search-based planning with latent world models (Zhou et al., 2025; Assran et al., 2025), our verifier ranks action chunk candidates by predicting delta progress without the need for a goal image. This makes it compatible with real-time VLA inference.

Architecture. We instantiate the verifier using the Joint-Embedding Predictive Architecture (JEPA) world model framework (Assran et al., 2025; Mur-Labadia et al., 2026), which consists of a pretrained visual encoder and an action-conditioned autoregressive predictor. The encoder com-

presses the current observation into a latent representation, while the predictor models how the latent state evolves conditioned on robot actions.

To adapt the pretrained world model to our setting, we introduce embodiment-specific state encoders and action encoders together with sub-chunk rollouts. For each embodiment, the encoders project the raw robot states and actions into the predictor embedding dimension. Each action chunk with horizon H is uniformly divided into M sub-chunks, enabling multi-step latent rollouts. This design provides denser supervision than predicting only the state at the end of the full action chunk, while reducing inference overhead and mitigating error accumulation compared with predicting a state after every individual action.

Building upon the latent world model, we introduce a delta progress prediction branch. Unlike latent state prediction, which models task-agnostic environment dynamics, delta progress prediction estimates how much the executed action sub-chunk advances the specified task. Consequently, it requires task-specific information beyond the current observation, robot state, and action. For example, the same state transition corresponds to positive progress for the task "closing the closet", but negative progress for "opening the closet". We therefore introduce a task-specific learnable query token to condition delta progress prediction.

Following the autoregressive paradigm, the predictor first imagines the next latent state $\hat{z}$ and then predicts the corresponding progress increment $\Delta\hat{p}$ for each input action sub-chunk. An attention mask prevents latent state prediction from attending to the task query. Otherwise, task-specific information could leak into the latent dynamics model, encouraging the predictor to exploit task semantics instead of learning task-agnostic environment dynamics. This design cleanly separates task-conditioned progress estimation from task-agnostic latent dynamics modeling, allowing the pretrained world model to be adapted for verification while preserving its original modeling objective.

Training. The verifier is jointly optimized for latent prediction and progress estimation, with progress estimation as the primary objective. Across the M action sub-chunks, the progress loss combines a Huber regression loss (Huber, 1964) with a pairwise ranking loss:

$$\mathcal{L}_{\text{prog}} = \sum_{i=1}^{M} \Bigg[\frac{1}{|\mathcal{B}_i|} \sum_{k \in \mathcal{B}_i} L_\delta\big(\Delta\hat{p}_i^{(k)}, \Delta p_i^{(k)}\big) + \frac{\lambda_{\text{rank}}}{|\mathcal{P}_i|} \sum_{(k,l)\in\mathcal{P}_i} \max\Big(0, \begin{matrix} -\big(\Delta\hat{p}_i^{(k)} - \Delta\hat{p}_i^{(l)}\big) \\ +\alpha\big(\Delta p_i^{(k)} - \Delta p_i^{(l)}\big) \end{matrix}\Big) \Bigg], \tag{8}$$

where $\Delta\hat{p}_i^{(k)}$ and $\Delta p_i^{(k)}$ denote the predicted and target progress for sample k at sub-chunk i, respectively. The sets $\mathcal{B}_i$ and $\mathcal{P}_i$ contain the batch samples and valid within-task sample pairs at sub-chunk i, respectively. The coefficient λ_{rank} balances regression accuracy and ranking consistency, while α controls the target-difference-dependent ranking margin. The Huber loss improves regression accuracy, while the rank loss preserves the relative ordering within tasks to avoid the trivial averaging solution. The absolute progress is labeled autonomously with Robometer (Liang et al., 2026a), and then the delta progress is calculated based on the result. The visual prediction loss is defined following V-JEPA 2-AC (Assran et al., 2025),

$$\mathcal{L}_{\text{vis}} = \sum_{i=1}^{M} \|\hat{z}_{t+i\cdot\frac{H}{M}} - z_{t+i\cdot\frac{H}{M}}\|_1. \tag{9}$$

The total training loss is computed as

$$\mathcal{L} = \mathcal{L}_{\text{prog}} + \lambda_{\text{vis}}\mathcal{L}_{\text{vis}}. \tag{10}$$

We use both teacher forcing and self-forcing (Huang et al., 2025) during the training stage to balance learning efficiency and inference distribution alignment.

Inference. At inference time, the predictor conditions exclusively on its previously predicted latent frames and autoregressively rolls out over the M sub-chunks of each candidate action chunk. The verifier thus produces M delta progress estimates for each candidate. The robot then executes the action chunk with the largest cumulative predicted progress, computed as the sum of these M estimates.

To keep verification practical for real-world deployment, three latency optimizations are applied cumulatively: (O1) reusing the KV cache of the verifier throughout the autoregressive rollout, (O2) optimizing the inference pipeline by overlapping independent computations, including the parallel generation of candidate action chunk samples and performing latent feature encoding in parallel with the action generation of the VLA policy, and (O3) PyTorch-level optimization, including pre-compilation.

4. Experiments

We organize our experiments around the core question: *how* the verifier should be designed. We first evaluate the mechanism in three settings: (1) real-world fine-grained manipulation tasks, validating the complete system under practical latency and hardware constraints, (2) RoboTwin 2.0 clean, studying the trade-off between inference efficiency and performance, and (3) RoboTwin 2.0 clean2random, showcasing the generalization ability of the verifier under randomized settings. We conduct ablation studies to assess the model design. We provide supporting experiments in Appendix C, a quantitative study of the proposed trigger, and Appendix B.2, experiments on LIBERO demonstrating policy-agnostic improvements across two base policies and 40 tasks.

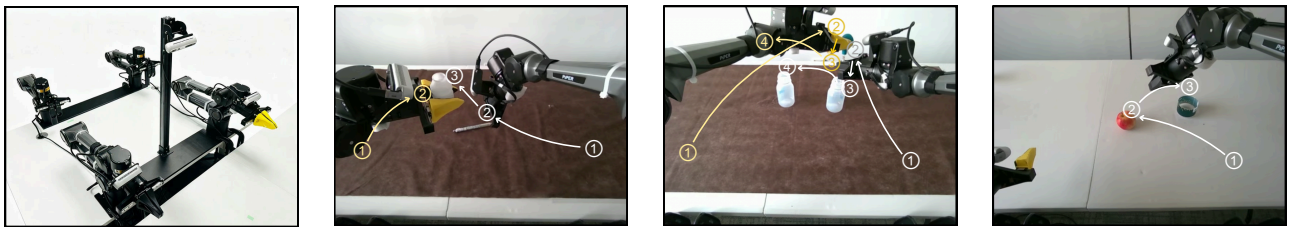


Figure 4. **Real-world experimental setup.** We conduct the real-world experiments on an AgileX PiperX dual-arm robot, and design three fine-grained tasks, namely, pen insertion, pipetting and apple placement.

Table 1. **Comparison of different methods across tasks and metrics.**

Method	Validation Metrics		Completion Score (%) ↑			
	Rank Loss ↓	Huber Loss ↓	Pen Insertion	Pipetting	Apple Placement	Average
$\pi_{0.5}$ (Intelligence et al., 2025)	–	–	45.0	52.5	55.0	50.8
+ RoboMonkey (Kwok et al., 2025)	1.19	0.37	49.0	55.5	63.5	56.0
+ GPC-RANK (Qi et al., 2026)	1.18	0.30	46.0	54.5	58.0	52.8
+ Ours	**1.09**	**0.25**	60.0	63.5	73.0	**65.5**

4.1. Real-world Experiments

Setup. We conduct real-world experiments on the AgileX PiperX robotic embodiment, equipped with one front camera (RealSense D455) and two wrist cameras (RealSense D435), as shown in Figure 4. To evaluate methods in different situations, we design three tasks with varying arm configurations (one single-arm and two dual-arm) and key challenges. The tasks are pen insertion (pick up the pen and insert it into the yogurt bottle), apple placement (pick up the apple and place it into the container), and pipetting (transfer liquid from the right bottle to the left with a pipette), shown in Figure 4. The detailed task configurations are provided in Appendix D.

Implementation Details. For each task, we collect 50 demonstration episodes to fine-tune the base policy. We then collect another 50 policy rollout episodes per task, label them using Robometer (Liang et al., 2026a), and train one verifier for all tasks in the environment on a single H800 GPU. The same protocol is used in the simulation experiments. Each method is evaluated over 50 trials per task in the real-world, RoboTwin 2.0, and LIBERO experiments. The test-time deployment is based on one RTX 4090 GPU, with the policy model and the verifier running on the same device. During testing, we set the trigger threshold to a certain value, where verification is turned on during contact-rich phases. We use $k = 2$ samples for Two-Axis Trigger and $K = 10$ samples for the verifier.

Metrics. We utilize three metrics: two validation losses and one closed-loop completion score. Specifically, *rank loss* measures the model's ability to rank the action chunk samples within one task. *Huber loss* measures how well the model regresses delta progress. *Completion score* reflects the system's closed-loop performance on the real-world tasks, where each stage is assigned a predefined score; details are provided in Appendix D.

Baselines. We compare against the following methods: (1) Base policy ($\pi_{0.5}$) with single sampling; (2) VLM-based verifier following the setting in RoboMonkey (Kwok et al., 2025), with the training objective adapted to delta progress estimation; (3) Pixel-level world model verifier following GPC-RANK (Qi et al., 2026).

Results. The results are shown in Table 1. Our method improves the real-world completion score by 14.7 percentage points, outperforming all baselines. Our three-level optimization reduces the latency to 326 ms per triggered decision (trigger latency excluded), shown in Figure 5. RoboMonkey performs competitively on tasks requiring only coarse-scale localization (*e.g.*, apple placement), but struggles with fine-grained manipulation, likely because its VLM lacks an explicit understanding of world dynamics. GPC-RANK achieves strong performance when estimating progress from real observations; however, its performance degrades on test-time rollout frames, indicating that the test-time generation quality becomes a bottleneck.

4.2. Simulation Results

To provide reproducible and quantifiable results, we evaluate the combination of the trigger and the verifier in simulation environments, including RoboTwin 2.0 (Chen et al., 2025) and LIBERO (Liu et al., 2023).

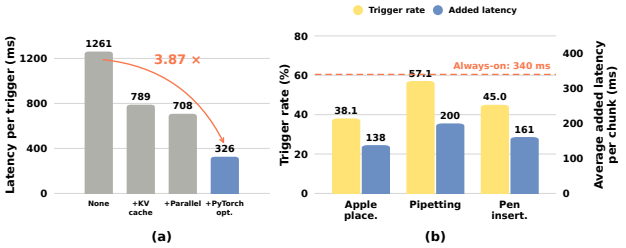


Figure 5. **Latency analysis. (a) Additional verification latency per triggered decision for 10 samples under successive optimizations.** The three optimization stages cumulatively reduce latency. KV caching provides the largest latency reduction. **(b) Trigger rate (left axis) and the resulting average added latency per chunk (right axis) on the three real-world tasks.**

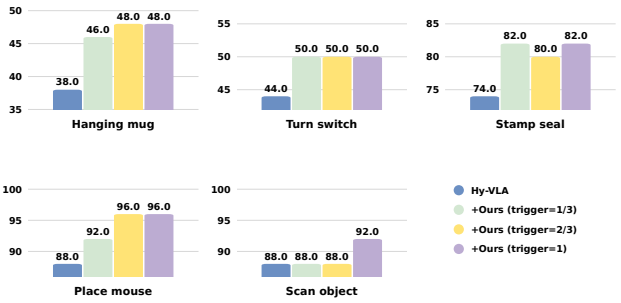


Figure 6. **Results on RoboTwin 2.0 Clean with Varying Trigger Thresholds.** For trigger $= r$, we adjust a general trigger threshold for all the tasks, so that approximately an r fraction of decision steps trigger verification.

Table 2. **Results on RoboTwin 2.0 Clean and Clean2Random with always-on verification.** Success rates are reported in percent.

Method	Mug	Switch	Seal	Mouse	Scan	Avg.
RoboTwin 2.0 Clean						
Hy-VLA	38.0	44.0	74.0	88.0	88.0	66.4
+ RoboMonkey	34.0	48.0	70.0	88.0	86.0	65.2
+ GPC-RANK	42.0	**50.0**	78.0	88.0	86.0	68.8
+ **Ours**	**48.0**	**50.0**	**82.0**	**96.0**	**92.0**	**73.6**
RoboTwin 2.0 Clean2Random						
Hy-VLA	16.0	50.0	72.0	80.0	**96.0**	62.8
+ RoboMonkey	22.0	44.0	78.0	86.0	94.0	64.8
+ GPC-RANK	22.0	50.0	**80.0**	82.0	90.0	64.8
+ **Ours**	**26.0**	**56.0**	78.0	**92.0**	**96.0**	**69.6**

RoboTwin 2.0. RoboTwin 2.0 is a benchmark for bimanual manipulation with both clean and randomized settings. For RoboTwin 2.0, we use Hy-Embodied-0.5-VLA (Zhang et al., 2026b) as the base policy. We evaluate our method on five fine-grained tasks that have relatively low baseline success rates. In the clean setting, we sweep a trigger threshold for all the tasks so that different fractions of decision steps trigger verification. Results show that (1) always-on verification improves the success rate on all five tasks; (2) the trigger could reduce computation while retaining much of the success-rate improvement, as shown in the first four tasks in Figure 6. In the Clean2Random setting, both the policy and verifier are trained exclusively on clean data and evaluated in randomized environments with always-on verification. Table 2 shows that our verifier can boost the base policy in randomized settings by 6.8 percentage points, showcasing generalization capability.

4.3. Ablation Study

In this section, we discuss the design of the verifier, and specifically answer the following question: Do *latent world modeling* and *pretrained latent dynamics* improve progress estimation? We conduct two sets of ablation studies. First, we assess the role of latent world modeling by removing the visual prediction objective. Second, we investigate whether pretraining the predictor on the general-purpose DROID dataset (Khazatsky et al., 2024) leads to better performance.

Table 3. **Ablation on world modeling and predictor pretraining.** Completion scores (%) are reported overall and per task. The visual encoder remains pretrained in all variants.

Method	Rank ↓	Huber ↓	Pen ↑	Pipetting ↑	Apple ↑	Avg. ↑
VeriVLA	**1.09**	**0.25**	**60.0**	**63.5**	**73.0**	**65.5**
w/o WM	1.21	0.26	52.0	55.5	58.0	55.2
w/o pretrain	1.17	**0.25**	54.0	61.0	63.5	59.5

4.3.1. Effect of World Modeling.

Does the test-time improvement stem from latent world modeling, or merely from a strong visual encoder? To answer the question, we implement a variant that predicts only delta progress, without jointly predicting future visual latents. Therefore, the variant degenerates into an autoregressive transformer without world modeling. We train the variant on the same three real-world tasks. As shown in Table 3, the completion score drops from 65.5% to 55.2%. These results suggest that without the dense world modeling objective, the model struggles to learn informative dynamics, leading to substantially weaker performance.

4.3.2. Effect of Predictor Pretraining

Our verifier builds on a frozen pretrained V-JEPA 2 encoder, which provides a rich visual prior learned from web-scale video data. In addition, the action-conditioned predictor is pretrained on 23k trajectories from the DROID dataset, offering a strong dynamics prior for robotic manipulation. To assess whether this predictor pretraining is beneficial for knowledge transfer, we train a variant without pretraining where the predictor is initialized randomly. In Table 3, the variant shows lower completion scores on real-world tasks, especially on the tasks with stronger generalization requirements, *e.g.* apple placement. In contrast, on the pipetting

task, where bottle locations remain almost fixed and generalization is less critical, the performance degradation is relatively minor.

5. Limitations and Future Work

A key limitation of repeated sampling and selection is the difficulty of steering a pretrained VLA model. Although the verifier can rank sampled action candidates, its effectiveness depends on the diversity and quality of the proposals generated by the underlying policy. The diversity of repeated sampling with different random seeds is still relatively limited. Another limitation is the use of a frozen visual encoder. While freezing the encoder reduces training cost and preserves pretrained representations, these representations are not explicitly optimized for manipulation progress estimation. They may overlook subtle changes in object configurations, contact conditions, or task completion that are essential for distinguishing promising actions from ineffective ones. Future work can investigate learning an encoder–predictor pair for progress estimation within a JEPA framework by considering progress estimation as an optimization goal.

6. Conclusion

We have introduced an adaptive test-time verification framework for exploiting the stochasticity of flow-based VLA policies. Our training-free two-axis trigger identifies critical states from within-chunk motion variation and cross-sample disagreement, invoking additional sampling only when it is likely to be useful. A JEPA-based latent world model then ranks candidate action chunks by jointly predicting action-conditioned future latents and delta task progress, avoiding pixel reconstruction and goal-image dependence. Experiments in both real-world and simulation demonstrate consistent gains across tasks, environments, and base policies. On three real-world tasks, our method increases the average completion score from 50.8% to 65.5% while adding 326 ms for sampling and verification per triggered decision, excluding the 14 ms trigger overhead. Ablation studies further confirm the benefits of latent dynamics modeling and pretraining. Taken together, these results establish adaptive latent verification as an effective and practical approach to improving frozen generative VLAs at deployment.

References

Agia, C., Sinha, R., Yang, J., Cao, Z., Antonova, R., Pavone, M., and Bohg, J. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *CoRL*, 2024. arXiv:2410.04640.

Assran, M., Bardes, A., Fan, D., Garrido, Q., Howes, R., Komeili, M., Muckley, M., Rizvi, A., Roberts, C., Sinha, K., Zholus, A., Arnaud, S., Gejji, A., Martin, A., Hogan, F. R., Dugas, D., Bojanowski, P., Khalidov, V., Labatut, P., Massa, F., Szafraniec, M., Krishnakumar, K., Li, Y., Ma, X., Chandar, S., Meier, F., LeCun, Y., Rabbat, M., and Ballas, N. V-jepa 2: Self-supervised video models enable understanding, prediction and planning, 2025. URL https://arxiv.org/abs/2506.09985.

Bai, Z., Gao, C., and Shou, M. Z. Evolve-vla: Test-time training from environment feedback for vision-language-action models, 2025. URL https://arxiv.org/abs/2512.14666.

Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al. π_0: A vision-language-action flow model for general robot control. In *RSS*, 2025a.

Black, K., Galliker, M. Y., and Levine, S. Real-time execution of action chunking flow policies. In *NeurIPS*, 2025b. arXiv:2506.07339.

Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., Ibarz, J., Ichter, B., Irpan, A., Jackson, T., Jesmonth, S., Joshi, N., Julian, R., Kalashnikov, D., Kuang, Y., Leal, I., Lee, K.-H., Levine, S., Lu, Y., Malla, U., Manjunath, D., Mordatch, I., Nachum, O., Parada, C., Peralta, J., Perez, E., Pertsch, K., Quiambao, J., Rao, K., Ryoo, M. S., Salazar, G., Sanketi, P. R., Sayed, K., Singh, J., Sontakke, S., Stone, A., Tan, C., Tran, H., Vanhoucke, V., Vega, S., Vuong, Q. H., Xia, F., Xiao, T., Xu, P., Xu, S., Yu, T., and Zitkovich, B. RT-1: Robotics Transformer for Real-World Control at Scale. In *Proceedings of Robotics: Science and Systems*, 2023.

Chen, J., Wang, K., Chen, K., Chen, S., Gao, F., Tang, W., Li, Z., Liu, W., Yao, Z., Li, B., Xu, Y., and Yu, C. Lawam: Latent world action models for efficient dynamics-aware robot policies, 2026a. URL https://arxiv.org/abs/2606.15768.

Chen, S., Yuan, J., Wang, J., and Chen, T. Trust your instincts: Confidence-driven test-time rl for vision-language-action models, 2026b. URL https://arxiv.org/abs/2606.29892.

Chen, T., Chen, Z., Chen, B., Cai, Z., Liu, Y., Li, Z., Liang, Q., Lin, X., Ge, Y., Gu, Z., Deng, W., Guo, Y., Nian, T., Xie, X., Chen, Q., Su, K., Xu, T., Liu, G., Hu, M., ang Gao, H., Wang, K., Liang, Z., Qin, Y., Yang, X., Luo, P., and Mu, Y. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation, 2025. URL https://arxiv.org/abs/2506.18088.

Chi, C., Feng, S., Du, Y., Xu, Z., Cousineau, E., Burchfiel, B., and Song, S. Diffusion Policy: Visuomotor policy learning via action diffusion. In *RSS*, 2023.

Fan, X., Lu, Y., Gao, S., Wu, X., Han, R., Li, M., and Zhao, H. Dreamavoid: Critical-phase test-time dreaming to avoid failures in vla policies, 2026. URL https://arxiv.org/abs/2605.11750.

Fei, S., Wang, S., Shi, J., Dai, Z., Cai, J., Qian, P., Ji, L., He, X., Zhang, S., Fei, Z., Fu, J., Gong, J., and Qiu, X. Libero-plus: In-depth robustness analysis of vision-language-action models, 2025. URL https://arxiv.org/abs/2510.13626.

Feng, X., Cheng, Y., Shi, C., Han, B., Yan, Y., Hong, Y., Tian, Z., and Jiang, L. Denoising tells when to replan: Denoising-variance adaptive chunking for flow-based robot policies. *arXiv preprint arXiv:2606.03847*, 2026.

Gu, Q., Ju, Y., Sun, S., Gilitschenski, I., Nishimura, H., Itkina, M., and Shkurti, F. Safe: Multitask failure detection for vision-language-action models. *Advances in Neural Information Processing Systems*, 38:40041–40076, 2025.

Hu, X., Liu, B., Liu, X., and Liu, Q. Adaflow: Imitation learning with variance-adaptive flow-based policies. *NeurIPS*, 2024.

Huang, X., Li, Z., He, G., Zhou, M., and Shechtman, E. Self Forcing: Bridging the train-test gap in autoregressive video diffusion. In *NeurIPS*, 2025.

Huber, P. J. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732.

Intelligence, P., Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Galliker, M. Y., Ghosh, D., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., LeBlanc, D., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Ren, A. Z., Shi, L. X., Smith, L., Springenberg, J. T., Stachowicz, K., Tanner, J., Vuong, Q., Walke, H., Walling, A., Wang, H., Yu, L., and Zhilinsky, U. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL https://arxiv.org/abs/2504.16054.

Jang, S., Kim, D., Kim, C., Kim, Y., and Shin, J. Verifier-free test-time sampling for vision-language-action models. In *ICLR*, 2026. arXiv:2510.05681.

Khazatsky, A., Pertsch, K., Nair, S., Balakrishna, A., Dasari, S., Karamcheti, S., Nasiriany, S., Srirama, M. K., Chen, L. Y., Ellis, K., et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.

Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., et al. OpenVLA: An open-source vision-language-action model. In *CoRL*, 2024.

Kwok, J., Agia, C., Sinha, R., Foutter, M., Li, S., Stoica, I., Mirhoseini, A., and Pavone, M. Robomonkey: Scaling test-time sampling and verification for vision-language-action models. In *CoRL*, 2025.

Kwok, J., Zhang, X., Xu, M., Liu, Y., Mirhoseini, A., Finn, C., and Pavone, M. Scaling verification can be more effective than scaling policy learning for vision-language-action alignment. *arXiv preprint arXiv:2602.12281*, 2026.

Liang, A., Korkmaz, Y., Zhang, J., Hwang, M., Anwar, A., Kaushik, S., Shah, A., Huang, A. S., Zettlemoyer, L., Fox, D., Xiang, Y., Li, A., Bobu, A., Gupta, A., Tu, S., Biyik, E., and Zhang, J. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons, 2026a. URL https://arxiv.org/abs/2603.02115.

Liang, Y., Wang, X., Wang, K., Wang, S., Peng, X., Chen, H., Chua, D. K. H., and Vadakkepat, P. Adaptive action chunking at inference-time for vision-language-action models, 2026b. URL https://arxiv.org/abs/2604.04161.

Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., and Stone, P. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS*, 2023.

Liu, C., Liu, Y., Wang, T., Zhuang, Q., Liang, J. C., Yang, W., Xu, R., Wang, Q., Liu, D., and Han, C. On-the-fly vla adaptation via test-time reinforcement learning, 2026. URL https://arxiv.org/abs/2601.06748.

Liu, Y., Ibn Hamid, J., Xie, A., Lee, Y., Du, M., and Finn, C. Bidirectional decoding: Improving action chunking via guided test-time sampling. In *ICLR*, 2025. arXiv:2408.17355.

Luo, H., Zhang, W., Feng, Y., Zheng, S., Xu, H., Xu, C., Xi, Z., Fu, Y., and Lu, Z. Being-h0.7: A latent world-action model from egocentric videos, 2026. URL https://arxiv.org/abs/2605.00078.

Maes, L., Lidec, Q. L., Scieur, D., LeCun, Y., and Balestriero, R. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels, 2026. URL https://arxiv.org/abs/2603.19312.

Mur-Labadia, L., Muckley, M., Bar, A., Assran, M., Sinha, K., Rabbat, M., LeCun, Y., Ballas, N., and Bardes, A. V-jepa 2.1: Unlocking dense features in video self-supervised learning, 2026. URL https://arxiv.org/abs/2603.14482.

Nakamoto, M., Mees, O., Kumar, A., and Levine, S. Steering your generalists: Improving robotic foundation models via value guidance. In *8th Annual Conference on Robot Learning*, 2024. URL https://openreview.net/forum?id=6FGlpzC9Po.

NVIDIA, Bjorck, J., Castañeda, F., Cherniadev, N., Da, X., Ding, R., Fan, L. J., Fang, Y., Fox, D., Hu, F., Huang, S., Jang, J., Jiang, Z., Kautz, J., Kundalia, K., Lao, L., Li, Z., Lin, Z., Lin, K., Liu, G., Llontop, E., Magne, L., Mandlekar, A., Narayan, A., Nasiriany, S., Reed, S., Tan, Y. L., Wang, G., Wang, Z., Wang, J., Wang, Q., Xiang, J., Xie, Y., Xu, Y., Xu, Z., Ye, S., Yu, Z., Zhang, A., Zhang, H., Zhao, Y., Zheng, R., and Zhu, Y. Gr00t n1: An open foundation model for generalist humanoid robots, 2025. URL https://arxiv.org/abs/2503.14734.

Pang, T., Suh, H. J. T., Yang, L., and Tedrake, R. Global planning for contact-rich manipulation via local smoothing of quasi-dynamic contact models, 2023. URL https://arxiv.org/abs/2206.10787.

Pertsch, K., Stachowicz, K., Ichter, B., Driess, D., Nair, S., Vuong, Q., Mees, O., Finn, C., and Levine, S. Fast: Efficient action tokenization for vision-language-action models, 2025. URL https://arxiv.org/abs/2501.09747.

Qi, H., Yin, H., Zhu, A., Du, Y., and Yang, H. Inference-time enhancement of generative robot policies via predictive world modeling. *IEEE Robotics and Automation Letters*, 2026.

Ross, S., Gordon, G. J., and Bagnell, J. A. A reduction of imitation learning and structured prediction to no-regret online learning, 2011. URL https://arxiv.org/abs/1011.0686.

Seligmann, F., Gospodinov, E., Dincer, E. U., and Neumann, G. Vla-fail: Efficient task failure detection for finetuned vision-language-action models. *arXiv preprint arXiv:2606.21386*, 2026.

Sobal, U., Zhang, W., Cho, K., Balestriero, R., Rudner, T. G., and LeCun, Y. Learning from reward-free offline data: A case for planning with latent dynamics models. In *Advances in Neural Information Processing Systems*, 2025.

StarVLA Community. Starvla: A lego-like codebase for vision-language-action model developing, 2026. URL https://arxiv.org/abs/2604.05014.

Sun, J., Zhang, W., Qi, Z., Ren, S., Liu, Z., Zhu, H., Sun, G., Jin, X., and Chen, Z. Vla-jepa: Enhancing vision-language-action model with latent world model, 2026. URL https://arxiv.org/abs/2602.10098.

Wu, Y., Tian, R., Swamy, G., and Bajcsy, A. From foresight to forethought: VLM-in-the-loop policy steering via latent alignment. In *Robotics: Science and Systems (RSS)*, 2025. arXiv:2502.01828.

Xu, C., Yu, E., Xuan, J., and Lu, J. Vgas: Value-guided action-chunk selection for few-shot vision-language-action adaptation, 2026. URL https://arxiv.org/abs/2602.07399.

Yang, L., Jiang, Z., Sheng, C., and Tang, Z. Guided action flow: Q-guided inference for flow-matching vision-language-action policies, 2026. URL https://arxiv.org/abs/2607.02092.

Zhang, H., Lu, Y., Wang, B., Kang, X., Kuo, Y.-L., Cheng, Z., Wang, M., and Jenkins, O. C. Foresight: Failure detection for long-horizon robotic manipulation with action-conditioned world model latents. *arXiv preprint arXiv:2606.23085*, 2026a.

Zhang, H., Xiang, L., Lin, H., Huang, Z., Wang, M., Zhong, D., Dong, Y., Wu, Y., Rao, Y., Zhang, D., He, W., Chen, L., Huang, K., Chen, J., Su, S., Yu, X., Wang, Z., Zhu, C., Teng, X., Guo, Y., Zhang, Y., Liu, Y., Wang, R., Lu, Z., Hu, H., and Zhang, Z. Hy-embodied-0.5-vla: From vision-language-action models to a real-world robot learning stack, 2026b. URL https://arxiv.org/abs/2606.14409.

Zhang, W., Terver, B., Zholus, A., Chitnis, S., Sutaria, H., Assran, M., Balestriero, R., Bar, A., Bardes, A., LeCun, Y., and Ballas, N. Hierarchical planning with latent world models, 2026c. URL https://arxiv.org/abs/2604.03208.

Zhao, C., Wang, Z., Li, H., Bao, S., Li, G., Feng, Y., Li, Y., Tang, J., and Zhang, J. Action draft and verify: A self-verifying framework for vision-language-action model, 2026. URL https://arxiv.org/abs/2603.18091.

Zhou, G., Pan, H., Lecun, Y., and Pinto, L. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *ICML*, 2025.

Zitkovich, B., Yu, T., Xu, S., Xu, P., Xiao, T., Xia, F., Wu, J., Wohlhart, P., Welker, S., Wahid, A., et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *CoRL*, pp. 2165–2183, 2023.

Appendix

A. Extended Related Work

In this section, we provide an extended review of prior work.

A.1. Test-time Verification

Existing methods differ in how the verifier is constructed. *Model-based verifiers* train a separate scoring model: RoboMonkey (Kwok et al., 2025) fine-tunes a 7B VLM on synthetic action preferences, CoVer (Kwok et al., 2026) argues that scaling such verifiers beats scaling policy learning, and V-GPS (Nakamoto et al., 2024) re-ranks samples with an offline-RL value function. *Model-free* methods instead read the signal off the policy itself, via masked-condition divergence (Jang et al., 2026) or proximity to the discrete action tokens (Zhao et al., 2026); both measure agreement rather than progress, and neither reasons about the physical consequences of a candidate. *World-model verifiers* close this gap by imagining outcomes, but do so in two stages: GPC-RANK (Qi et al., 2026) and DreamAvoid (Fan et al., 2026) synthesize future pixels before scoring them, while FOREWARN (Wu et al., 2025) rolls out latents and then aligns a per-task finetuned VLM to narrate them. VeriVLA keeps the world model but removes the second stage, reading delta progress directly from predicted latents. Combined with a two-axis action-based trigger, verification is invoked adaptively.

Table 4. **Comparison of test-time verification methods.**

Method	Trigger	World Modeling	Verification Pipeline	Prediction Target
RoboMonkey	always-on	×	one-stage	action preference
FOREWARN	always-on	✓	two-stage	latent, VLM narration
GPC-RANK	always-on	✓	two-stage	pixel, reward
ADV	always-on	×	one-stage	proximity to discrete action tokens
MG-Select	always-on	×	one-stage	masked-condition divergence
DreamAvoid	vision-based	✓	two-stage	pixel, delta progress
VeriVLA	action-based	✓	one-stage	latent, delta progress

A.2. Other Test-time Enhancement

In addition to test-time verification, we provide an overview of other existing test-time enhancement methods, including test-time RL, adaptive action chunking, and online monitoring.

Test-time training and RL. Test-time training methods, such as EVOLVE-VLA (Bai et al., 2025) and TT-VLA (Liu et al., 2026), perform online policy optimization during deployment using dense reward signals. Rather than relying on external reward estimation, T^2VLA (Chen et al., 2026b) derives intrinsic rewards from the policy's own confidence to enable self-supervised adaptation. These methods update policy parameters and therefore require reward access, gradient computation, and tolerance for transient regression during adaptation.

Adaptive action chunking. A parallel line of work asks not *which* action to take but *how* to emit it. BID (Liu et al., 2025) resamples action chunks under backward coherence and forward contrast to trade off consistency against reactivity. RTC (Black et al., 2025b) targets the latency gap by generating the next chunk while the current one executes, freezing the actions guaranteed to run and inpainting the remainder. AAC (Liang et al., 2026b) adaptively sets the chunk size from action entropy, and DVAC (Feng et al., 2026) triggers replanning from denoising variance. The underlying intuition traces back to AdaFlow (Hu et al., 2024), which links the conditional variance of the flow-matching loss to ODE discretization error and reduces to a one-step generator when the action distribution is unimodal. Our trigger shares this variance-based intuition, but these methods reshape the execution or decoding schedule of a single trajectory and never assess whether the emitted actions make task progress; our trigger instead gates a verifier that can reject and replace a chunk rather than merely smoothing or resizing it.

Online monitoring and failure detection. Runtime monitors aim to predict failure before it becomes irrecoverable. Sentinel (Agia et al., 2024) splits failures into erratic modes, detected by statistical temporal consistency of action-chunk distributions (STAC), and task-progression modes, detected via VLM video QA; the two detectors together catch 18% more failures than either alone. SAFE (Gu et al., 2025) shows that VLA internal features carry task-generic success/failure information and trains a lightweight scalar failure predictor on them, using conformal prediction for a calibrated time-varying threshold;

VLA-FAIL (Seligmann et al., 2026) pairs a last-layer Mahalanobis distance with an action-chunk consistency term to remove the need for failure data. Closest to our setting, Foresight (Zhang et al., 2026a) extracts action-conditioned latents from V-JEPA 2-AC and predicts a per-timestep failure score, showing that world-model features generalize across policies better than policy-internal ones.

B. Training Details and LIBERO Experiments

B.1. Hyperparameters

We list the training hyperparameters in Table 5.

Table 5. **Training hyperparameters and resource configuration.** The table specifically shows the real-world adaptation experiments. The actual training epochs and duration can largely depend on the scale of the dataset.

Hyperparameter	**Value**
Learning rate	5×10^{-4}
Weight decay	0.04
Batch size	32
Optimizer	AdamW
JEPA loss weight	1
Rank loss weight	1
Crop size	256×256
Patch size	16
Tubelet size	2
Number of sub-chunks	5
Resource	**Specification**
GPU model	1 NVIDIA H800 (80 GB)

B.2. LIBERO Experiments

We evaluate our method on LIBERO (Liu et al., 2023), using $\pi_{0.5}$ and StarVLA-GR00T (StarVLA Community, 2026) as the base policies. As shown in Table 6, the results indicate that our method improves performance on LIBERO-10 suite, while maintaining strong performance on other suites. A single verifier supports all 40 tasks and improves performance across both policies.

Table 6. **LIBERO results.** Our method boosts the base policy, especially on LIBERO-10, the task suite that has not been saturated.

Method	Spatial	Object	Goal	10	Avg.
$\pi_{0.5}$	**98.8**	98.2	98.0	92.4	96.9
+ RoboMonkey	98.6	**99.0**	97.6	**94.0**	97.3
+ GPC-RANK	97.6	96.8	97.8	89.0	95.3
+ Ours	**98.8**	98.8	**98.6**	**94.0**	**97.6**
StarVLA-GR00T	98.0	**99.4**	96.4	91.0	96.2
+ RoboMonkey	98.2	**99.4**	96.0	91.6	96.3
+ GPC-RANK	**98.6**	99.2	96.6	92.4	96.7
+ Ours	98.4	99.0	**97.8**	**93.6**	**97.2**

C. Trigger Experiments

C.1. Evaluation Metrics

Our trigger aims to detect critical task states or states where the policy is uncertain. Since there is no ground truth for when to trigger, we propose two metrics in simulation environments to evaluate how well the trigger value aligns with our setting: (1) *Status AUC* is the AUC-ROC between the trigger value and a rule-based *critical label*. The label is a binary indicator obtained by thresholding temporal proximity to *critical moments*, which are read from the simulator and defined as the moments when task-related objects switch between contact and non-contact, with the gripper or another task-related object.

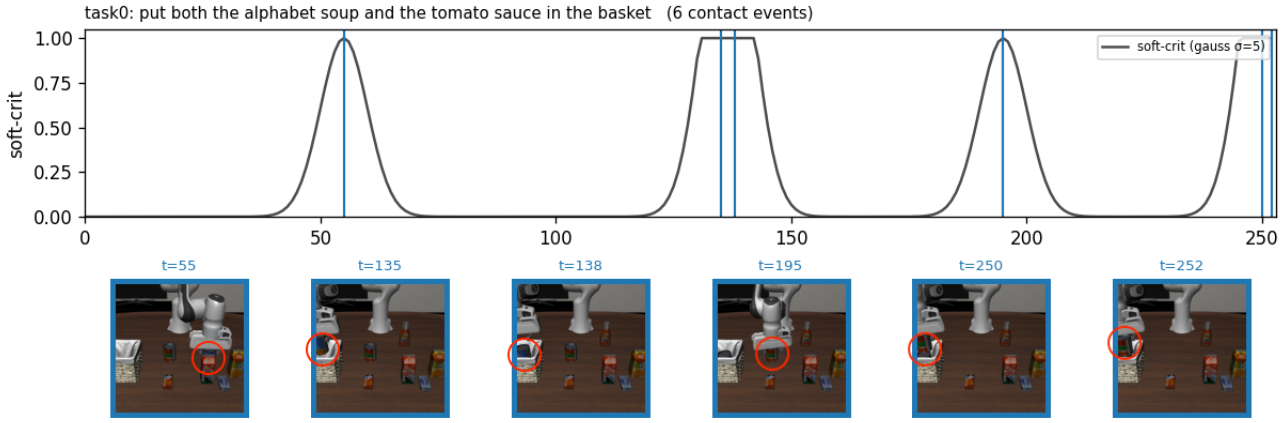


Figure 7. **Critical Index and Critical Moments in LIBERO.**

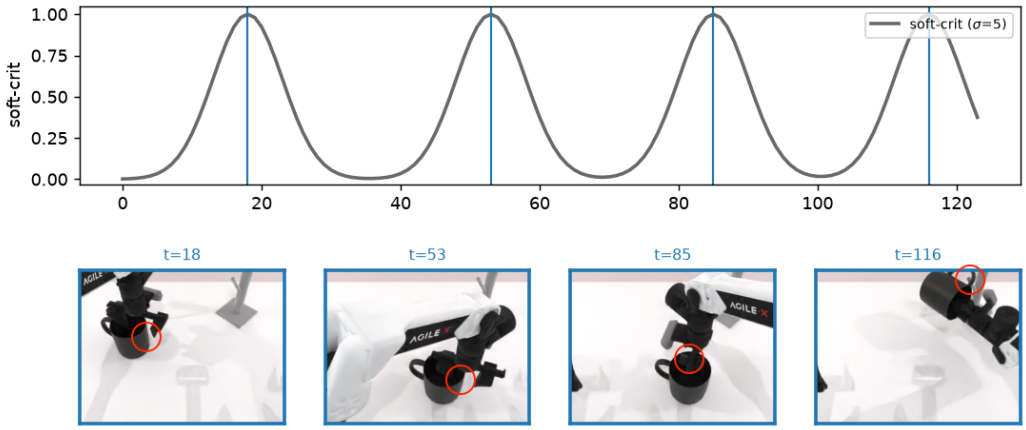


Figure 8. **Critical Index and Critical Moments in RoboTwin.**

This measures how well the trigger identifies contact-relevant stages that we hypothesize to be most challenging. It serves as a fair metric, since the criterion is computed solely from simulator states rather than the visual states or policy actions that are used by triggers, and it avoids manual annotation. The critical-label construction is detailed in Section C.2. (2) *Variance Correlation* measures Spearman's ρ between the trigger value and the variance of candidates from repeated sampling. This metric indicates how well the trigger activates when the policy is most uncertain, and when the VLA provides varying candidates.

C.2. Critical-label Construction

To evaluate whether a trigger signal at a state $V_{\text{trigger}}(s_t)$ identifies moments that genuinely require more compute, we construct a reference label $\text{crit}(t) \in [0, 1]$ that is independent of the action or vision. We find the *critical moments* from privileged simulator state (object/gripper pose, joint angles, contact) using a rule: we mark frames where gripper-object or object-object contact state flips (grasp / release) for any task-relevant object. For each support moment, we assign a Gaussian kernel function. The *critical index* for each frame is then computed as the sum of all overlapping kernel contributions, clipped to a maximum of 1.

After labeling an episode with the continuous critical index, we compute the trigger value for each chunk $a_{t:t+H}$ and take the maximum critical index within that window. Thresholding the latter at 0.5 yields the binary label, and the trigger value serves as the score; we then compute the ROC curve and the Status AUC. We visualize two tasks from LIBERO and RoboTwin for reference:

Table 7. **Trigger performance analysis.** Status AUC and Variance Correlation are defined in Section C.1. Each method uses k (defined in Section 3.3) action chunk samples, and consequently introduces additional latency per action chunk.

Method	Status AUC ↑	Variance Correlation ↑	k / add. latency (ms) ↓
random	0.487	–	–
DINO	0.702	0.630	**0 / 0**
ACE	0.588	0.716	2 / 14
T_{temp}	0.704	0.789	**1 / 0**
T_{cross}	0.694	0.877	2 / 14
Ours	**0.727**	**0.887**	2 / 14

Table 8. **Completion score breakdown for the three real-world tasks.**

Task	Stage 1	Score	Stage 2	Score
Pen Insertion	Grasp the pen	0.5	Insert the pen into the yogurt bottle	0.5
Pipetting	Aspirate water from the right bottle	0.5	Dispense water into the left bottle	0.5
Apple Placement	Grasp the apple	0.5	Place the apple into the container	0.5

C.3. Baselines

We compare the proposed Two-Axis Trigger against representative prior methods as well as its individual components: (1) A lightweight MLP trained on frozen DINOv2 features, as a representative method for a vision-based baseline (Fan et al., 2026). We train the model with the signals read from the simulator, and evaluate it together with the other methods on the perturbation set from LIBERO-Plus (Fei et al., 2025). (2) Action chunk entropy (ACE) (Liang et al., 2026b). Although originally designed for adaptive action chunking rather than the trigger, it provides an action-based baseline. (3) The temporal trigger T_{temp} and cross-sample variance trigger T_{cross}, components of our Two-Axis Trigger.

C.4. Results

As shown in Table 7, our method, without any training, achieves the best Status AUC of 0.727 and Variance Correlation score of 0.887, indicating that the proposed Two-Axis Trigger aligns well with our requirements. The vision-based DINO baseline achieves a competitive Status AUC of 0.702 but a lower Variance Correlation of 0.630. The action-based baselines achieve higher Variance Correlation, while their Status AUC is below or comparable to that of the vision-based baseline. Our trigger uses two samples and adds 14 ms of latency per decision step.

D. Real-world Setting

D.1. Tasks

We conduct experiments on three real-world tasks, pen insertion, pipetting, and apple placement. Table 8 defines the completion score of each task.

D.2. Computational Cost

We report the inference computational cost in Table 9, including latency and GPU usage.

Table 9. **Latency and GPU memory usage breakdown.**

Latency	
Component	Latency (ms)
VLA policy	82
Two-Axis Trigger (k=2)	14
Additional sampling ($K-k$=8 more candidates)	80
Verifier forward (all K=10 candidates)	246
Total latency (deactivated)	96
Total latency (activated)	422
Additional latency (verification only)	326
GPU Memory Usage	
Component	GPU Memory (GB)
VLA policy	9.8
Verifier forward (all K=10 candidates)	8.7
Combined GPU memory	18.5